Project specification document (prototype)

App name: EventVault

Tech stack: flutter for mobile version and front end React JS + backend Django for web version and PostgreSQL as database.

Project Description:

Build a ticket reservation platform for events (concerts (need more events for examples) etc.). The core idea here is to encourage a shift from physical tickets to digital ones by providing a portal where event organizers can come and manage the tickets for their events.

I. Comprehensive Functionality List

- **Multi-Platform Access:** Cross-platform mobile app (Flutter) for buyers/agents and a web dashboard (React) for complex organizer tasks.
- **Multi-Tier Ticketing:** Ability to set up separate pricing, quantities, and perks for Standard, VIP, VVIP, etc.
- **AI Template Engine:** Prompt-to-image generator allowing organizers to design unique background art for tickets.
- **QR Security Ecosystem:** Generation of a structurally unique QR code overlaid onto the custom ticket design.
- **Agent Invitation System:** Organizers can add specific phone numbers/emails of gate staff, granting them "Scan-Only" access to verify tickets.
- **Escrow & Refund Protocol:** Automated holds on ticket payouts, with instant automated rollbacks to customer wallets/payment methods if a cancellation occurs.
- **Expanded Event Types (Brainstorming Examples):** Beyond concerts, expand to **Tech Conferences/Hackathons** (with badge generation), **Exclusive Club Nights** (with table bookings), **Sports Matches** (with section/seat assignments), and **Private Gala Dinners** (with RSVP guest lists).
- **Offline Ticket Verification:** Verification agents' apps should cache encrypted ticket hashes locally so they can scan and validate QR codes even if the venue has terrible internet connection or network drops.

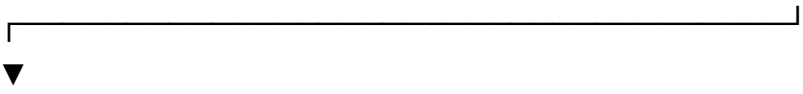
- **Anti-Scalping Dynamic QR:** The QR code on the user's mobile app can subtly regenerate every 15 seconds based on a time-based token (TOTP). This prevents users from simply screenshotting a VIP ticket and sending it to 5 friends.

Ideas still to be studied:

- **Souvenir "NFT-style" Digital Memorabilia:** Once a ticket is scanned at the venue, the downloaded PDF/Image in the user's profile turns into a "Past Experience" digital trophy, preserving the custom AI artwork as a digital keepsake.
- **The "In-App Resale" Marketplace:** If a buyer can no longer attend, they can list their ticket back on your platform at face value. The app invalidates their QR code and generates a fresh one for the new buyer, keeping all transactions secure and within your escrow loop.

a. Dynamic Ticket Creation Flow:

[Organizer Auth] ➔ [Create Event Details] ➔ [Define Ticket Tiers (VIP/Standard)]



[Choose Artwork Source: Upload Own OR Generate via AI] ➔ [Inject System QR Code Zone] ➔ [Publish & Save]

Customers select an event -> views all events info -> selects ticket category he wants to pay for -> customer pays for ticket and his ticket is generated with unique QR code -> customer downloads ticket

For the concern of how to prevent any fraud or scams we will implement the **Escrow Logic** that will permit a fund retention for all tickets paid such that if the event is cancelled the users are refunded.

. **The 48-Hour Validation Window:** The organizers are supposed to validate the event completion on the platform with funds sent to them within 48 hrs prior proper verification on if the event actually happened. Instead of just trusting the organizer's click, "validation" requires a two-factor verification:

* **Proof of Event:** A minimum percentage (e.g., 50%+) of sold tickets must be scanned by the invited verification agents at the venue.

* **No Dispute Trigger:** If no major fraud/cancellation reports are filed by users within 24 hours post-event, funds are automatically cleared for release to the organizer within the 48-hour mark.

b. The "Gazo Concert" Use Case & Escrow Logic

Using a real-world scenario like the financial losses from the cancelled Gazo concert perfectly justifies our architecture.

- **The Problem:** Traditional ticketing platforms pay organizers upfront. If an event is cancelled at the last minute, organizers might have already spent the money on production, making refunds incredibly difficult to recover.
- **The Enhanced Logic:** 1. **The Fund Lock:** Ticket sales accumulate in an escrow account managed by your backend (via APIs like Stripe, Flutter wave, or local mobile money APIs like MTN/Orange MoMo).

II. Competitive Analysis

To position our app uniquely, here is how it compares to two major players in the ticketing space:

Eventbrite

- **What it does:** The industry standard for event discovery, ticketing, and registration. It allows organizers to create events, manage sales, and scan tickets via an app.
- **How your app is different: * Escrow Security:** Eventbrite typically releases funds to organizers payouts before the event happens (or on a rolling schedule). If an organizer cancels and runs with the money, Eventbrite has to chase them down, often leaving fans in limbo. Your **Escrow Logic** completely shifts the trust dynamic by securing the funds until event completion.
    - **AI Customization:** Eventbrite offers standard text-and-logo ticket layouts. Your integration of an **AI image generation tool** for custom ticket templates gives organizers unique branding power right inside the platform.

Resident Advisor (RA) / DICE

- **What they do:** Popular platforms focused heavily on nightlife and concerts, utilizing secure, dynamic QR codes within their apps to prevent scalping and fraud.
- **How your app is different:**
    - **Open Ticket Design Flexibility:** While DICE locks tickets completely inside their closed app ecosystem (you can't download a PDF), your app allows organizers to upload custom artwork or generate it via AI, blending security (unique QR codes) with a personalized collectible aspect.
    - **Agent-Specific Portals:** Your explicit focus on inviting verification agents streamlines gate control for independent, localized events without forcing the main organizer account to be logged in on multiple devices.

Idea to be discussed

Either Allow the PDF download with a unique, static QR code, but implement a strict backend check so that once that specific QR code is scanned *once* by a verification agent, it is instantly deactivated in the database. This keeps your development simple, fulfils your document's requirements, and stops basic duplicate entry fraud.

Or Save the Dynamic QR Code (and the restriction of PDF exports) for a "High-Security Event Tier" that organizers can toggle on if they are managing massive celebrity concerts prone to heavy scalping.

Since the plan is to provide an offline dynamic QR code

How Offline Dynamic QR Codes Work

Instead of the mobile app constantly asking your Laravel/Django backend for a new QR code, the app and the server calculate the code independently using **Time** and a shared **Secret Key**.

Here is the exact structural logic of how it works offline:

1. The Initial Setup (Online)

When the customer buys the ticket, the backend generates a unique, highly secure string called a **Secret Key** for that specific ticket.

- This secret key is downloaded and securely stored inside the Flutter app's local storage (using a package like flutter_secure_storage) right after successful payment.

2. Generating the QR Code (100% Offline)

When the user arrives at the event venue and opens the app, the app doesn't need network bars. It performs a local mathematical calculation:

$$\text{Current Unix Time} \div 15 \text{ seconds} = \text{Current Time Step}$$

The app hashes the **Secret Key** together with the **Current Time Step** using a crypto algorithm (like SHA-256). The result is a short, unique string of characters. The Flutter app instantly turns this string into a QR code on the screen. Because the Unix clock on phones is accurate even without internet, the app can change this QR code every 15 seconds completely offline.

3. Verification at the Gate (Offline or Online)

When the verification agent scans the ticket with their phone:

- **If the agent is Online:** The agent's app sends the scanned string to your backend. The backend runs the exact same math calculation using the ticket's secret key and the current time. If the strings match, the ticket is valid!
- **If the agent is Offline:** For the agent to verify it offline, the agent's device would need to pre-download a synchronized list of ticket secret keys before the gate opens. The agent's phone then performs the math locally to check if the QR code is valid.

The Catch: Device Clock Synchronization

The only technical rule for this to work is that **the user's phone clock must be relatively accurate**. If a user manually changes their phone's settings to 3 hours in the past, the generated QR code will be incorrect, and

the scanner will reject it. (Most modern smartphones sync their time automatically via network providers, so this is rarely an issue, but it's an edge case to keep in mind).